

Couple Tracker (SoLuna)

The premise

Couples usually keep their photos, videos, and relationship memories in normal phone folders or cloud drives. Because of that, it can be hard to organize them and remember what date or event they are connected to. Couple Tracker gives two people a private shared space where they can save, organize, and look back at their memories together.

Who is this for?

Couple Tracker is for couples who want one private place for their relationship photos, videos, and memories. Instead of just using regular phone folders, Google Drive, or other storage apps, they can use an app that's actually made for organizing relationship memories and important events.

Core features (MVP)

#	Feature	Status	Flutter Pieces	Estimate
1	Shared Memory Storage	Keep	ListView.builder, Card, model class, image widgets	5H
2	Event/Date Filtering	Keep	TextField, filter controls, ListView.builder	2H
3	Anniversary/Event showcase	Keep	ListView.builder, image widgets, date filtering	3H
4	Memory/Event Details	Keep	Navigator.push, Scaffold, Card	3H
5	Scrapbook Designs	Stretch	GridView, custom layouts, image widgets	6H

Stretch goals

- Scrapbook-style layouts for photos and memories.
- Yearly relationship recaps that show important memories from previous years.
- Daily relationship status or check-in features.

How my app saves data

If two different people install my app, should they see the same data? (yes / no, and why)

If two different people install the app and belong to the same couple, they should both be able to see the same shared photos/memories. This is important because the main point of Couple Tracker is for both people

to add and look at the same collection of photos, videos, and events.

Roughly how many records does my app hold in a realistic week of use?

A normal week of using the app could probably have around 20–40 new memory records.

My choice:

I would use Supabase because the app needs shared data between two users.

Why this one and not the others, in two or three sentences. Name the tradeoff you accepted.

The downside is that Supabase needs more setup compared to local storage, but I think it's necessary because shared access is one of the main purposes of the app. Just saving everything locally wouldn't really work for two people using the same couple account.

What I save, concretely

Thing	Fields	Where saved
User	userId, name, email	Supabase database
Couple	coupleId, user1Id, user2Id, relationshipStartDate	Supabase database
Folder	folderId, coupleId, name	Supabase database
Memory	memoryId, coupleId, title, description, date, mediaUrl, media	Supabase database + storage
Event	eventId, coupleId, eventName, eventDate, eventType	Supabase database

One thing I want to add that the course did not teach

I want to add notifications for important dates, like anniversaries or saved events. I would use a Flutter notification package for this.

How my project runs when someone else opens it

I am keeping the device_preview wrapper: yes.

My app runs in a browser with flutter run -d web-server, with every main screen reachable: yes, except features requiring device-only functionality.

Anything requiring real hardware degrades to sample data instead of crashing: yes.

The project will not contain real personal photos, passwords, or private user information. Required API keys or credentials will be kept out of the public repository.

Screens

1. Login Screen

This is where the user can enter or access their account.

2. Main Screen

This shows recent memories, upcoming events, and relationship recaps.

3. Memory/Folders Screen

This displays the saved photos and videos. Users can also filter their memories by date or event.

4. Memory Details Screen

This shows one specific photo or video, along with its description, date, and related event.

Risks

One risk is handling the app's data and storage. Since the app needs to share data between two people, I need to make sure each couple only sees the correct memories. My first step will be making a small Supabase test database and checking if I can successfully save and retrieve one memory record.

Another risk is that photos and videos can be much bigger than normal text data. My first step will be to upload one sample image, save its storage URL inside a memory record, and display it again in Flutter before trying to support multiple uploads.

What changed and why

Section	Prelim said	Now says	Why
Screens	Main, folders, settings, scrapbook	Account, On This Day, folders, Couple Dates, detail, settings	Makes the problem more specific.
Features	File sharing, notifications, filtering	Shared memories, filtering, events, details	Matches what is shown in the design.
Data	Current day, statuses, how long an event has been	Users, couples, memories, events, settings with fields	Removes vague data descriptions.
Scrapbook	Part of original screen list	Stretch goal	Too much to actually consider

How my project runs when someone else opens it

I am keeping the device_preview wrapper.

The app can run in a browser using flutter run -d web-server, and the main screens should all be reachable, except for features that need device-only functionality.

Anything that requires real hardware will use sample data instead of crashing the app.

The project will not include real personal photos, passwords, or private user information. Any required API keys or credentials will also be kept out of the public repository.